

Balancing Optimizer, Capacity, and Regularization for ResNet Classification on Tiny ImageNet

Lauren Bolte
University of California, San Diego
lbolte@ucsd.edu

Abstract

The ResNet family of convolutional neural network models offers a unique opportunity to explore the impact of transfer learning on image classification task performance. Performance remains sensitive to hyperparameter choices despite having a stable architectural backbone. In this paper, we present a systematic empirical study of transfer learning on the Tiny ImageNet dataset, focusing on the effects of optimizer selection (AdamW vs. Stochastic Gradient Descent), deep layer unfreezing strategies, and regularization techniques on ResNet models 18 and 34. We find that when the entire network backbone is frozen, AdamW and SGD do not differ in their effect on performance due to the task's reduction to linear classification on fixed features. Validation accuracy improves significantly as deeper layers are unfrozen. However, the issue of overfitting is introduced as model capacity increases. We mitigate this concern with regularization techniques and extend our analysis to a deeper ResNet-34 network. The deeper architecture shows the highest validation accuracy at nearly 60%, with SGD outperforming AdamW on the moderate-scale image classification tasks.

1. Introduction

Image classification is a continuing challenge in the machine and deep learning community, with state-of-the-art architectures being developed at an exceptional rate. Images are a particularly difficult type of data to work with because of the variety of features that models must be able to identify and classify, despite similarities between objects that will ultimately be part of different classes. Convolutional Neural Networks (CNNs) have achieved success in these image classification tasks, particularly with residual architectures such as ResNet, which enable the effective training of deep models through skip connections [2]. Training these architectures is highly computationally expensive and time consuming. In order to study the hyperparameters and architecture styles best suited for image classification, datasets like TinyImage Net have

been developed to offer a more realistic option for small-scale fine-tuning experiments.

Pre-trained models like ResNet also provide an opportunity for hyperparameter experimentation without the need to design an architecture from scratch. However, when applying these models to smaller-scale image datasets, overfitting and generalization challenges become especially apparent.

This experiment aims to discover which fine-tuning specifications within the ResNet-18 pretrained architecture can increase test performance while controlling for overfitting. We use the TinyImage Net dataset, which offers a more complicated challenge than the CIFAR-10 dataset typically experimented with in deep learning coursework, but is more computationally feasible for the purposes of this experiment than the full size Image Net data. We perform this in-depth analysis through transfer learning, rather than reinitializing the weights and training a model from scratch. Transfer learning establishes a relationship between solutions to the target task and previous tasks. Knowledge from a source task is utilized in learning towards completion of a target task [2]. This study uses transfer learning to improve convolutional network performance on limited data. We hypothesize that an increase in flexibility through progressive layer unfreezing will improve validation accuracy but lead to overfitting in later training stages. Additionally, we expect Stochastic Gradient Descent (SGD) to outperform AdamW in generalization as model capacity increases.

2. Methods

2.1 Architecture

The introduction of residual networks to the field of deep learning offered an alternative to the process of simply adding more layers to a network to create a deeper network, which experiences gradient explosion and reduced accuracy when enough layers are added. Accuracy is oversaturated and degrades quickly [1]. A deep residual learning framework addresses this degradation problem by introducing skip connections, or residual connections in the case of ResNet-18, which skip one or more layers and perform identity mapping. This process doesn't require extra

parameters or computational complexity, avoiding the need for each layer to learn an entirely new representation. This method follows the assumption that residual identity representations will be easier to learn and optimize than the original function [1]. It preserves information and stabilizes during training [2]. In this experiment, we utilize the strong backbone of ResNet-18 to trial different hyperparameters and avoid overfitting on the compact TinyImage Net dataset.

The ResNet-18 model is 18 layers deep, containing 17 convolutional layers, a fully-connected layer, and a softmax layer that performs a classification task [1]. Alterations made in this experiment are on the hyperparameters, while the backbone of the model is maintained for stability.

2.2 Data and Problem

In order to determine if optimizer choice changes ResNet-18 performance on TinyImage Net data, a systematic empirical study was conducted using transfer learning techniques. In terms of performance, we focus on the convergence behavior and generalization capabilities as an overall metric for each hyperparameter alteration. The three key factors being changed for training configuration comparisons are optimizer choice (SGD and AdamW), flexibility through the progressive unfreezing of deeper layers, and regularization strategies such as dropout and weight decay. The final hyperparameters are then applied to the ResNet-34 architecture, which is similarly defined by the presence of residual connections but much deeper with 34 layers instead of 18. We can then examine the impact of architectural depth on our hyperparameter choices.

2.3 Hyper-Parameters

The hyper-parameters chosen for fine-tuning in this experiment do not impact the layer backbone of the model, but instead change the way the model performs when presented with training and validation data. The two main groups of comparison between model architectures are defined by the optimizer choice: Stochastic Gradient Descent (SGD) and Adaptive Moment Estimation with Weight Decay (AdamW). The role of an optimizer is to adjust the weights and biases of a model during the learning process. SGD updates parameters based on individual samples and can introduce noisy learning dynamics, while AdamW offers faster convergence and robustness with per-parameter learning rates with first and second moment estimates of the gradients [5]. The choice we make alters the behavior of the model's convergence, how quickly it learns the training data, and generalization to unseen validation data.

In addition to optimizer choice, we vary model flexibility. Controlled flexibility through progressive

unfreezing of deeper network layers allows the model performance to change, often positively, but creates a potential for overfitting. To counter this issue, we utilize regularization techniques such as dropout in the classifier layers and weight decay in the optimizer. Dropout prevents overfitting by deactivating a portion of the neurons during training, forcing the model to learn more robust features [6]. It is applied to the fully connected layers and is not necessary during the validation phase. Weight decay restricts the values that parameters can take, shrinking the weight parameter towards zero during each step of training [7]. The combination of these hyperparameter variations allows for an evaluation of the trade-offs between model convergence behavior, flexibility, and generalization performance.

3. Experiment

Our experiment is performed in steps based on optimizer choice, which layers in the ResNet-18 backbone are frozen, and whether dropout present in the classifier. The metric used to determine if the model is performing better or worse than before fine-tuning is validation accuracy and both the training and validation loss over the course of the training epochs. Each step of fine-tuning is completed with a graph of the loss to visualize whether overfitting is a valid concern.

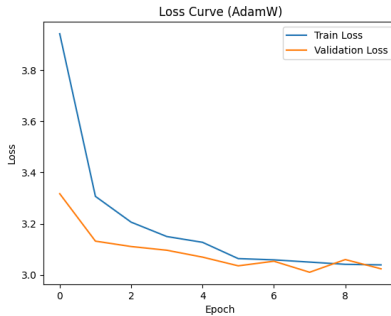
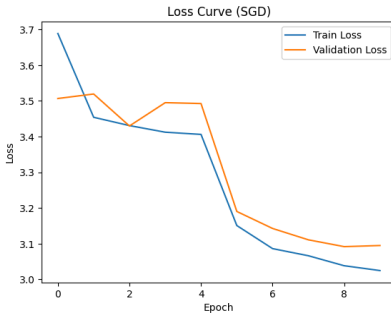
3.1 Pre-processing

Before initializing the fine-tuning process, the TinyImage Net data was normalized to offer faster convergence and stable training. Normalization is especially useful on the ResNet-18 model because it is trained on the full-size ImageNet, so utilizing the official ImageNet mean and standard deviation allows for effective transfer learning.

3.2 Training Procedure

The core training loop of ResNet-18 remains unchanged throughout the study. Each portion of the experiment is defined by modifications to the optimizer choice, progressive unfreezing of deeper layers, and the inclusion of dropout and weight decay in the classifier. The initial configuration uses a fully frozen backbone, where only the final classification layer is trained. To increase model flexibility, deeper layers are progressively unfrozen. First, layer 4 is unfrozen, followed by both layers 3 and 4.

Training duration is adjusted as model flexibility increases. Early configurations are trained for 10 epochs, while later configurations, particularly those with unfrozen layers 3 and 4, are trained for 30 epochs. A dummy scheduler is introduced in later stages to allow additional flexibility in the training process.

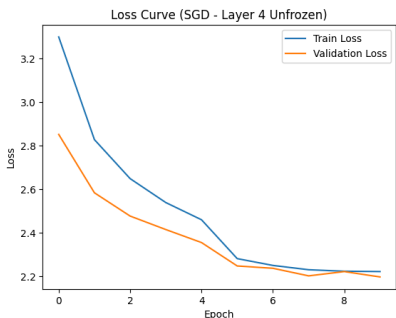
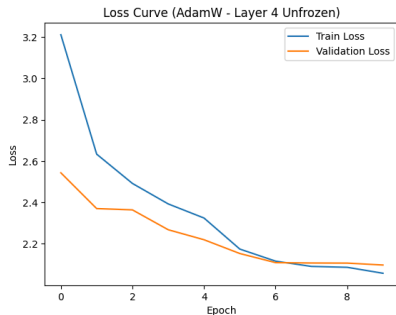


To address potential overfitting associated with increased model capacity, regularization techniques are introduced. A dropout layer is added to the classifier, and weight decay of 1×10^{-4} is applied within the optimizer. Finally, model capacity is

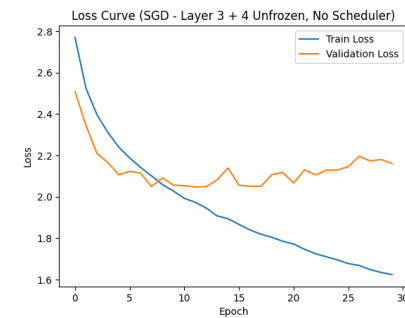
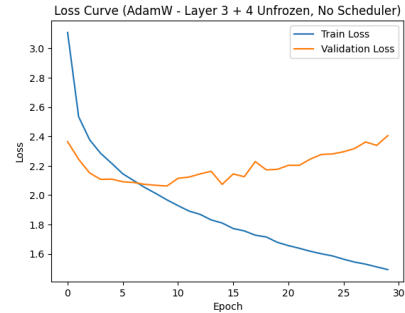
increased by replacing ResNet-18 with ResNet-34. For this deeper architecture, a lower learning rate is used to maintain stable training while keeping the same regularization strategies.

4. Results

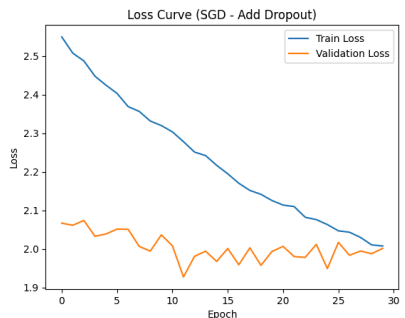
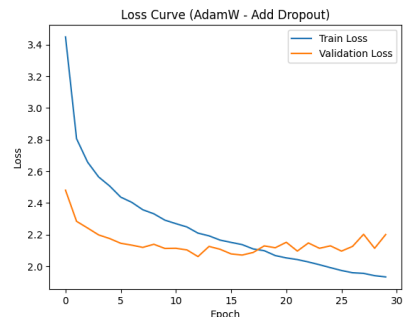
When the backbone is fully frozen, optimizer choice has minimal impact on performance. AdamW and SGD have nearly identical validation accuracy at approximately 34%, indicating that the task reduces to multiclass logistic regression on fixed pretrained features. Although AdamW converges faster and SGD provides more stable updates, these differences do not affect final performance in this setting [Figure 1].



Increasing model flexibility through layer unfreezing leads to substantial improvements in validation accuracy. Unfreezing layer 4 increases accuracy to approximately 50%, with both optimizers exhibiting similar behavior [Figure 2].



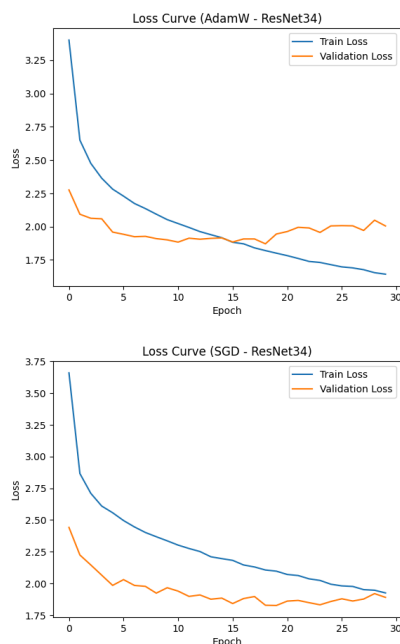
Further unfreezing of layers 3 and 4 results in additional performance gains, with SGD outperforming AdamW (53.16% vs. 51.38%). However, this configuration introduces overfitting, characterized by decreasing training loss alongside plateauing or



increasing validation loss [Figure 3]. AdamW shows slightly earlier overfitting compared to SGD.

The introduction of regularization techniques, including dropout and weight decay, helps stabilize training but does not significantly improve peak validation accuracy, which remains around 52% for both optimizers [Figure 4].

The highest performance is achieved using the deeper ResNet-34 architecture. Both optimizers benefit from increased model capacity, with SGD achieving the best validation accuracy (57.02%) compared to AdamW (56.84%). Overfitting is still present but occurs later in training (approximately 10–20 epochs), suggesting that earlier stopping could further improve generalization [Figure 5].



5. Discussion

This analysis demonstrates that transfer learning on the Tiny ImageNet data using ResNet models is strongly influenced by model flexibility, optimizer choice, and regularization techniques. Optimizer has a minimal impact in low-capacity settings, but SGD generalizes better in later stages as flexibility increases. Progressive layer unfreezing significantly improves model performance on unseen data, but overfitting becomes an issue. This problem is mitigated through regularization strategies such as dropout in the classifier block and weight decay to provide stability. The final extension of our analysis with the architectural depth of ResNet-34 yields the best performance overall. This suggests that deeper

pretrained models are better suited for image classification tasks on this scale. Visualizing our results reveals a plateau in validation loss in later epochs, suggesting that an earlier stopping strategy and learning rate scheduling could further improve generalization.

6. References

- [1] Alom, M., Taha, T. M., Yakopcic, C., et al. (2019). A state-of-the-art survey on deep learning theory and architectures. *Journal of Medical Systems*. Available at <https://link.springer.com/article/10.1007/s10916-019-1475-2>
- [2] Hosna, A., Merry, E., Gyalmo, J., Alom, Z., Aung, Z., and Azim, M. A. (2022). Transfer learning: a friendly introduction. *Journal of Big Data*, 9(1):102. Available at <https://pmc.ncbi.nlm.nih.gov/articles/PMC9589764/>
- [3] He, K., Zhang, X., Ren, S., and Sun, J. (2016). Deep residual learning for image recognition. In *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*. Available at <https://ieeexplore.ieee.org/document/7780459>
- [4] Roboflow (n.d.). ResNet-18 explained. Available at <https://blog.roboflow.com/resnet-18/>
- [5] TechRxiv (n.d.). Visualizing optimization dynamics: A comparative analysis of Adam vs SGD for non-linear regions. Available at <https://www.techrxiv.org/users/1026925/articles/1387033-visualizing-optimization-dynamics-a-comparative-analysis-of-adam-vs-sgd-for-non-linear-regions>
- [6] Zhang, A., Lipton, Z. C., Li, M., and Smola, A. J. (n.d.). Dive into deep learning: Dropout. Available at https://d2l.ai/chapter_multilayer-perceptrons/dropout.html
- [7] Zhang, A., Lipton, Z. C., Li, M., and Smola, A. J. (n.d.). Dive into deep learning: Weight decay. Available at https://d2l.ai/chapter_linear-regression/weight-decay.html#sec-weight-decay